Podcasting 2.0 and Clojure

State of support, where the holes are, and how a Clojure library family could be built
Prepared for Simon Gray
2026-09-20

1. Summary
2. What Podcasting 2.0 consists of
3. Support today
4. Where the holes are, and why
5. Libraries relevant to a Clojure effort
6. Does a Clojure library make sense?
7. Proposed architecture
8. Building the apps
9. Cloud syncing of OPML and playback state
10. Facts to verify before committing
11. Suggested sequence

Sources

1. Summary

- **Podcasting 2.0** is the `podcast` RSS namespace (about 30 tags) plus open infrastructure: the Podcast Index directory with its free API and downloadable database, and Podping for near-real-time feed-change notifications.
- **Support is two-tiered.** Mainstream apps (Apple, Spotify, Pocket Casts, Overcast, AntennaPod) have adopted a few tags, mostly transcripts and chapters. A ring of roughly 15 smaller "2.0 apps" implements most of the namespace, including value-for-value payments. Hosting platforms are well ahead of apps: most independent hosts emit `guid`, `locked`, `transcript`, `chapters`, `funding`, `person` and send Podpings.
- **The holes** are value-for-value outside the niche apps, live items, social interaction, alternate enclosures, location, publisher and podroll feeds. The causes are structural: platform incentives, App Store rules around payments, spec churn, format fragmentation, and the lack of shared libraries outside JavaScript.
- **Libraries.** JavaScript has de-facto reference parsers (podcast-partytime, feedsmith). Python and Dart have partial ones. The JVM has nothing: Rome has no podcast-namespace module. Clojure has nothing at all, for the namespace or for the Podcast Index API.
- **A Clojure library makes sense**, as one repository publishing a few small artifacts around a `.cljc` core: feed model and parser/emitter, companion formats (chapters, transcripts, OPML, guid), Podcast Index client, a JVM-only Podping consumer, and a sync engine. Value-for-value should be an optional, separate artifact.
- **Apps.** Build the web app as a ClojureScript PWA on a Clojure backend first. For iPhone, wrap the same ClojureScript core in Expo/React Native (shadow-cljs) when you need background downloads, CloudKit or an App Store presence.
- **iCloud sync.** Apple Podcasts' own data is not accessible. CloudKit is the only iCloud route that reaches both an iOS app and a web app (via CloudKit JS). Model sync as two documents, an OPML subscription list and a playback-state document, behind a pluggable backend, so iCloud, Nextcloud/WebDAV or a gpodder-style server can be swapped.

A note on sourcing: this report was compiled from my own knowledge of the ecosystem (current to mid-2026) and a small number of live checks on 2026-09-20. Items I could not confirm live are marked *(unverified)*. Before committing to a design, verify the handful of facts flagged in section 10.

2. What Podcasting 2.0 consists of

2.1 The namespace

The namespace URI is `https://podcastindex.org/namespace/1.0`, conventionally prefixed `podcast:`. It is maintained in the open at `github.com/Podcastindex-org/podcast-namespace`, with rendered docs at `podcasting2.org/docs/podcast-namespace`. Tags were introduced in "phases"; the table groups them by purpose instead.

Group	Tags	Level	Notes
Identity and control	`guid`, `locked`, `txt`, `block`, `updateFrequency`, `license`	channel (license also item)	`guid` is a UUIDv5 of the feed URL and is the stable identity everything else hangs on. `locked` prevents feed hijacking by hosts.
Episode enrichment	`transcript`, `chapters`, `soundbite`, `season`, `episode`, `trailer`, `person`, `location`, `image`	mostly item; `person`, `location`, `trailer`, `image` also channel	`transcript` and `chapters` are the most widely adopted tags. `images` (comma-separated srcset) was replaced by `image` in 2025. `location` was revised in 2025 (adds `rel`, `country`, `geo`).
Distribution	`alternateEnclosure` (+ `source`, `integrity`), `medium`, `remoteItem`, `podroll`, `publisher`, `contentLink`	item and channel	`medium` turns a feed into music, video, audiobook, etc. `remoteItem` references items in other feeds by guid and underpins playlists, music, and publisher feeds.
Money	`funding`, `value` (+ `valueRecipient`), `valueTimeSplit`	channel and item	`value` is keysend-centric (`type="lightning" method="keysend"` with node recipients). Lightning address (`lnaddress`) recipients are documented alongside node and Hive examples on the current docs site. `valueTimeSplit` redirects splits for a time range, e.g. featured music.
Live and social	`liveItem`, `socialInteract`, `chat`	channel (`liveItem`); item (`socialInteract`); both (`chat`)	`liveItem` is an item with a status and start/end time. `socialInteract` points at an ActivityPub, Nostr, Bluesky or similar root post. Both depend on other infrastructure to be useful.

2.2 Podcast Index

- A directory of about four million feeds, run by Podcast Index LLC (Adam Curry, Dave Jones), funded by donations and value-for-value. Non-profit in practice, no listener tracking.
- **API** at `api.podcastindex.org/api/1.0`. Free; you register for a key and secret. Each request carries `X-Auth-Key`, `X-Auth-Date` (Unix seconds) and `Authorization` = hex SHA-1 of key + secret + date, plus a required `User-Agent`. Endpoint groups: `search` (byterm, bytitle, byperson), `podcasts` (byfeedid, byfeedurl,

byguid, byitunesid, bytag, trending, dead), `episodes` (byfeedid, byfeedurl, byguid, byid, live, random), `recent` (episodes, feeds, newfeeds, soundbites), `value`, `categories`, `stats`, `hub/pubnotify`, `add`, and `apple` (an iTunes lookup replacement).

- **Database dump.** The full feed table is published as a SQLite file (multi-GB) for download from podcastindex.org. This is the cheapest way to get a realistic test corpus of feed URLs and metadata and to build local search.
- **Docs and code**: `podcastindex-org.github.io/docs-api` (OpenAPI available as `pi_api.yaml`). Official example code exists for several languages; there is no official JVM client.

2.3 Podping

Podping replaces polling with notifications. Hosts (or a relay at `podping.cloud`) write small `custom_json` operations to the Hive blockchain containing feed URLs and a reason (`update`, `live`, `liveEnd`). Consumers stream blocks from any Hive API node and filter for the `podping` id. The reference consumer is `podping-hivewatcher` (Python); Podcast Index itself is the largest consumer. Most independent hosts send Podpings; Apple, Spotify and the big enterprise hosts do not. WebSub remains the only mainstream alternative and is mostly used by Google/YouTube.

2.4 Podcast Standards Project

Formed in 2023 by hosts and app makers (Buzzsprout, Transistor, Captivate, Pocket Casts, Overcast and others). Its PSP-1 specification codifies a baseline of RSS 2.0 plus iTunes tags plus a small set of `podcast:` tags (roughly `guid`, `locked`, `transcript`, `funding`, `person`, `chapters`; treat the exact list as *(unverified)*). It matters because it is the vehicle through which mainstream apps have adopted namespace tags without committing to the whole of Podcasting 2.0.

3. Support today

3.1 Apps

Yes = supported, Partial = limited or only some formats, No = not supported, ? = not confirmed.

App	Platform	transcript	chapters	funding	person	value (V4V)	liveItem	socialInteract	Sync
Apple Podcasts	iOS/macOS	Yes (SRT/VTT ingested since 2024, else Apple auto-generates)	Partial (embedded ID3/M4A; JSON chapters ?)	No	No	No	No	No	iCloud, private
Spotify	all	? (own auto transcripts)	Partial (own)	No	No	No	No	No (own comments)	own
YouTube Music	all	No	No	No	No	No	No	No	own
Pocket Casts	iOS/Android/web	Yes (2024)	Yes (JSON + embedded)	?	No	No	No	No	own account
Overcast	iOS	?	Partial (embedded only)	No	No	No	No	No	own

App	Platform	transcript	chapters	funding	person	value (V4V)	liveItem	socialInteract	Sync
Castro	iOS	?	Partial	No	No	No	No	No	iCloud
Podcast Addict	Android	Yes	Yes	Yes	Yes	No	Partial	No	own/gpodder
AntennaPod	Android (open source, Kotlin)	Yes (since 3.4, 2024)	Yes (JSON)	Yes	No	No	No	No	gpodder.net / Nextcloud
Fountain	iOS/Android	Yes	Yes	Yes	Yes	Yes (own wallet)	Yes	Partial (Nostr)	own
Podverse	iOS/Android/web (AGPL, React Native)	Yes	Yes	Yes	Yes	Yes	Yes	Yes	own account
CurioCaster	web PWA	Yes	Yes	Yes	Yes	Yes	Yes	Partial	own
Castamatic	iOS	Yes	Yes	Yes	Yes	Yes	Yes	Yes	own
Podcast Guru	iOS/Android	Yes	Yes	Yes	Yes	Yes	?	?	own
TrueFans	web	Yes	Yes	Yes	Yes	Yes	Yes	Yes	own
Breez, LN Beats, Wavlake	wallet-first apps and music	Partial	Partial	Partial	Partial	Yes	No	No	none/own

Two patterns stand out. First, the tags that reached the mainstream are exactly the ones that improve a single app's listening experience with no coordination (transcripts, chapters). Second, no mainstream app syncs through anything a third party can reach; every one uses a private account or, for Apple and Castro, a private iCloud container.

The live list of apps and features is maintained at `podcastindex.org/apps` (the old `newpodcastapps.com` now redirects there).

3.2 Hosting platforms

Host	guid / locked	transcript	chapters	funding / person	value	Podping	Notes
Buzzsprout	Yes	Yes	Yes	Yes	Yes	Yes	PSP member; broad support
Transistor	Yes	Yes	Yes	Yes	Partial	Yes	PSP member
Captivate	Yes	Yes	Yes	Yes	Yes	Yes	PSP member
RSS.com	Yes	Yes	Yes	Yes	Yes	Yes	
Podhome	Yes	Yes	Yes	Yes	Yes	Yes	Built around the namespace; also `liveItem`, `podroll`, `publisher`
Castopod (open source, PHP)	Yes	Yes	Yes	Yes	Yes	Yes	Native ActivityPub, so `socialInteract` works end to end
Blubrry	Yes	Yes	Yes	Yes	Yes	Yes	
Castos, Spreaker, Podbean	Yes	Partial	Partial	Partial	Partial	Partial	

Host	guid / locked	transcript	chapters	funding / person	value	Podping	Notes
Libsyn	Yes	Partial	?	?	No	?	
Acast, Megaphone, Spotify for Creators, Omny	Partial (`guid` at most)	No	No	No	No	No	Enterprise hosts ignore the namespace

Consequence for a library author: real-world feeds mix every generation of tag, including deprecated forms (`images`, old `location` attributes, `value` blocks with only node recipients), and the enterprise hosts produce the bulk of listening hours with almost none of the new tags.

3.3 Adoption in numbers

Podcast Index publishes counts per tag in its stats and on the “Podcasting 2.0” board. Order-of-magnitude picture (exact figures move weekly and are *(unverified)* here): `guid` and `locked` are in a majority of actively updated feeds because hosts emit them automatically; `transcript` and `chapters` are in the low hundreds of thousands of feeds; `value` blocks are in the low tens of thousands; `liveItem`, `socialInteract`, `alternateEnclosure` and `location` are each in the thousands or fewer.

4. Where the holes are, and why

Hole	Why it exists
Value-for-value in mainstream apps	Apple and Spotify have their own subscription products and take a cut; Lightning payments would bypass them. App Store rules permit wallets only for enrolled organisations, so V4V apps either run custodial wallets (Fountain) or integrate a third-party wallet (Alby, Breez). Keysend is being deprecated by some node software, which forced the spec to add Lightning-address recipients, and that migration is unfinished.
Transcript fragmentation	Four formats (SRT, VTT, JSON, HTML), inconsistent speaker labels and language tags. Apple’s auto-transcripts reduce the incentive for creators to publish their own.
Chapters	JSON chapters and embedded ID3/M4A chapters coexist; apps pick one. Images and URLs in chapters are rarely rendered.
`liveItem` and `chat`	Only useful if the app also consumes Podping (for timely status) and can play a live stream; both are heavier lifts than the tag itself.
`socialInteract`	Requires the app to talk ActivityPub, Nostr or Bluesky, each with its own identity model. Only Castopod closes the loop on the hosting side.
`alternateEnclosure`	Hosts do not produce multiple encodings, so apps have nothing to consume; chicken and egg.
`location`, `person`, `publisher`, `podroll`	Metadata without a killer feature in the UI; adopted by hosts, ignored by most apps. `location` was redesigned in 2025, which reset partial implementations.
Spec churn	Tag renames (`images` to `image`), attribute redesigns and phase-based addition of tags mean every parser needs an active maintainer. Outside JavaScript, few libraries kept up.
No conformance suite	The namespace repo has example feeds and an XSD but no test corpus with expected parse results. Each implementation re-discovers the edge cases.

Hole	Why it exists
Sync	Podcasting 2.0 does not define subscription or playback sync. gpodder.net is operationally fragile, and the Open Podcast API (`openpodcastapi.org`, specs at `github.com/OpenPodcastAPI/api-specs`) is still being drafted with no widely deployed server.

5. Libraries relevant to a Clojure effort

Only the host platforms Clojure runs on, or can wrap, are listed.

Ecosystem	Library	What it covers	Relevance
JavaScript	`podcast-partytime` (`@podverse/podcast-partytime` fork)	The de-facto reference parser. Grew out of Podcast Index's own aggregator, used by Podverse. Reports which "phases" a feed uses.	The best specification-by-implementation for the parser. Usable directly from ClojureScript on Node or React Native, but its output shape is idiosyncratic.
JavaScript	`feedsmith` (v3.0 released 2026-09-18)	TypeScript parser and generator for RSS, Atom, RDF, JSON Feed and OPML with iTunes, Podcast Index, Spotify and other namespaces; normalises prefixes.	Actively maintained, typed, good model for a data-oriented design. Could be wrapped from CLJS instead of writing a parser for the JS target.
JavaScript	`podcastdx-client`, `podcast-index-api`	Podcast Index API clients.	Simple; the auth header is the only non-trivial part.
JavaScript	`@getalby/sdk` (NWC, Nostr Wallet Connect)	Wallet API including `pay_keysend`, `pay_invoice`, WebLN-compatible provider.	The pragmatic way to add V4V to a web or React Native app.
Python	`pod2gen`, `python-podcastindex`, `podping-hivewriter/hivewatcher`	Feed generation with namespace support, API client, Podping writer/reader.	Reference for Podping consumption logic.
Dart	`podcast_search`	iTunes and Podcast Index search, feed parsing with chapters, transcripts, persons.	Only relevant if you choose ClojureDart/Flutter for iPhone.
JVM	Rome + rome-modules	Modules exist for iTunes, MediaRSS, GeoRSS, Slash, Google Base, OpenSearch, SLE, SSE, Yahoo Weather. **No podcast-namespace module.**	Rome's module API is awkward and its parser is strict; not worth extending.
JVM	AntennaPod's feed parser (Kotlin, app-internal)	Transcript, chapters, funding, guid.	Not published as a library.
JVM	jsoup (`Parser.xmlParser()`)	Lenient XML/HTML parsing.	The fallback you want for malformed feeds.
Clojure	`clojure.data.xml`, `remus`, `feedparser-clj`, `clj-rss`	StAX parsing with namespace support (CLJS via DOMParser, no streaming); Rome wrappers; plain RSS generation.	No Clojure library covers the podcast namespace, the Podcast Index API, Podping or OPML with podcast conventions.

The JVM gap is real: every JVM podcast app parses feeds with hand-rolled code, and nothing on Maven Central or Clojars targets the namespace.

6. Does a Clojure library make sense?

Yes, with two conditions.

1. **It must be `.cljc`-first.** The value of Clojure here is one data model and one set of pure functions for feeds, chapters, transcripts, OPML and sync merging, shared by a JVM server, a browser app and a React Native app. A JVM-only library would serve the crawler and nobody else.
2. **It must be several small artifacts, not one.** The concerns have different platform needs and different dependency weights: streaming XML and Podping belong on the JVM; wallet payments drag in secp256k1 and Nostr; CloudKit is platform-specific. One repo, one version line, several deployable artifacts (the pattern used by lambdaisland and metosin libraries).

What it should *not* try to be: a podcast app framework. Player, downloads, storage and UI are app concerns and differ per platform.

7. Proposed architecture

7.1 Artifacts and layering

```
podcast.core.feed      (.cljc)  RSS 2.0 + iTunes + podcast namespace: data model, parse, emit,
validate
podcast.core.formats   (.cljc)  chapters JSON, transcripts (SRT/VTT/JSON/HTML), OPML, podcast:guid,
dates
podcast.index          (.cljc)  Podcast Index API client: pure request builders + HTTP adapters
podcast.podping        (.clj)   Hive stream consumer, JVM only
podcast.sync           (.cljc)  subscription + playback-state model, merge, backend protocol, adapters
podcast.v4v            (.cljc)  optional: value block resolution, boost/stream payment via NWC
```

Dependency direction is strictly downward: `formats` and `index` depend on `core.feed`'s data model; `sync` depends on `formats` (OPML) but not on `index`; `podping` and `v4v` depend only on `core.feed`.

7.2 Data model

- Plain maps with namespaced keys, one namespace per source: `:rss/title`, `:itunes/author`, `:podcast/guid`, `:podcast/transcripts`. Unknown elements are preserved under `:xml/extra` so nothing is lost on round trip.
- Malli schemas for every entity (feed, item, transcript ref, chapter, value block, live item, person). Malli gives validation, coercion, generators for tests and JSON Schema export for free. Keep the schemas in a separate namespace so a ClojureDart consumer can use the maps without Malli.
- Identity: `:podcast/guid` if present, else the computed UUIDv5 from the feed URL (namespace `ead4c236-bf58-58c6-a2c6-a6b28d128cb6`, scheme stripped, trailing slashes removed). Episode identity: item `guid`, falling back to enclosure URL. Every other artifact keys on these.

7.3 Parsing strategy

- Parse into a generic XML tree (`{:tag :attrs :content}`) through a small adapter protocol, then map that tree to the data model in pure `.cljc` code. Adapters: `clojure.data.xml` (JVM StAX, streaming, lazy per item), `DOMParser` in the browser, `@xmldom/xmldom` or `fast-xml-parser` on React Native, and the `xml` package if ClojureDart is ever added.
- Real feeds are frequently invalid XML (bad entities, control characters, wrong encoding declarations). On the JVM, parse strictly first and fall back to jsoup's XML parser on failure; record which path was used so the crawler can report feed health.

- A **tag registry** maps qualified element names to handler functions plus metadata (level, cardinality, deprecated-by). Spec churn then becomes a data change: adding `podcast:image` while still reading `podcast:images` is one registry entry with a normalisation step, and the same registry drives the emitter and the validator.
- Dates: RSS dates are RFC 822 in theory and anything in practice. Ship a lenient, platform-independent parser returning epoch milliseconds; do not depend on `java.time` formatters or `js-joda` in the core.
- Testing: build a fixture corpus from the Podcast Index database dump (a few thousand feeds stratified by host) and snapshot the parsed output. This is the conformance suite the ecosystem lacks and it is what makes the library trustworthy.

7.4 Podcast Index client

- Pure functions produce a request map (URL, headers, query); a thin adapter executes it with hato or clj-http on the JVM and `fetch` on JS. SHA-1 via `java.security.MessageDigest` and `goog.crypt.Sha1` (synchronous, works in browser, Node and React Native).
- The key and secret must never ship in a web client. The web app calls the Clojure backend, which holds the credentials and caches responses. For a mobile app the same proxy is the honest design; many apps embed keys, but the terms discourage it.
- Respect the API's expectations: a descriptive `User-Agent`, caching, and use of `recent`/Podping instead of polling `episodes/byfeedid` in a loop.

7.5 Crawler and Podping (JVM)

- Conditional GET with `ETag` and `If-Modified-Since`, gzip, redirect tracking (permanent redirects update the stored URL), per-host politeness and timeouts.
- Podping consumer: a Java 11 `WebSocket` or hato websocket to a Hive API node, streaming blocks, filtering `custom_json` ops with id `podping` and its variants, and emitting `{:url :reason :timestamp}` events onto a `core.async` channel or a callback. Keep the Hive specifics isolated; if Podcast Index ever offers a plain HTTP push, only this namespace changes.

7.6 Sync engine

- Two documents: `subscriptions.opml` (OPML 2.0; one `outline` per feed with `type="rss"`, `xmlUrl`, `text`, plus a `podcast:guid` attribute in the Podcast Index namespace) and `state.edn`/`state.json` (per-episode position, played flag, queue order, per-feed settings), each entry carrying a last-modified timestamp and a device id.
- Merge is last-writer-wins per entry, with an additive rule for subscription lists (a removal is stored as a tombstone with a timestamp so it beats an older addition). This is enough for one person's devices and avoids CRDT machinery.
- `SyncBackend` protocol: `fetch-doc`, `put-doc` with an optimistic version token, `list-versions`. Adapters, in order of value: CloudKit (native record store via an Expo module on iOS, CloudKit JS on the web), WebDAV/Nextcloud, gpodder-compatible API (nextcloud-gpodder or a self-hosted server), Dropbox and Google Drive app-folder APIs, and later the Open Podcast API when it stabilises.

7.7 Value-for-value (optional artifact)

- Resolve the effective value block for an item (item block overrides channel block; `valueTimeSplit` applies for a time range; `remoteItem` blocks are fetched through the Podcast Index `value` endpoint).
- Pay through Nostr Wallet Connect using the Alby SDK on JS (browser and React Native); on the JVM, either call an LND/CLN REST API or leave payments to the client. Boostagram metadata goes in TLV record 7629169 (bLIP-10) for keysend; Lightning-address recipients use LNURL-pay with the same JSON as a comment. Keep this artifact separate: it pulls in secp256k1 and WebSocket relays and is the part most exposed to App Store review.

7.8 Main dependencies

Concern	JVM	JS (browser, Node, React Native)
XML	`org.clojure/data.xml`, `org.jsoup/jsoup` (fallback)	`DOMParser` in browsers; `@xmldom/xmldom` or `fast-xml-parser` on RN/Node
HTTP	`hato` (Java 11 client, HTTP/2, websockets) or `clj-http`	`fetch`; `lambdaisland/fetch` for a Clojure-flavoured wrapper
JSON	`metosin/jsonista` or `cnuernber/charred`	`js/JSON`
Schemas	`metosin/malli`	`metosin/malli`
Hashing	`java.security.MessageDigest`	`goog.crypt.Sha1`
Async	`core.async` or `funcool/promesa` (works on both)	`promesa`
Serialisation for storage and sync	`cognitect/transit-clj`	`cognitect/transit-cljs`
App-side state	Datalevin or SQLite via `next.jdbc` (server)	Datascript persisted to IndexedDB/OPFS (web) or `expo-sqlite`/MMKV (RN)
Payments (optional)	LND/CLN REST clients	`@getalby/sdk` (NWC), `nostr-tools`

8. Building the apps

8.1 Web app

A ClojureScript single-page app (Reagent/re-frame, UIx or Helix; shadow-cljs) served by a Clojure backend (reitit + ring, or Pedestal) is the natural fit and reuses everything in the `.cljc` core. The backend does what a browser cannot or should not: crawl and cache feeds (bypassing CORS and hiding the Podcast Index credentials), consume Podping, run search over a local copy of the index, and host a sync backend. Ship the front end as an installable PWA from day one: manifest, service worker, Media Session API for lock-screen controls, and Cache/OPFS storage for offline episodes.

8.2 iPhone

Option	Shared code with the web app	Effort	Trade-offs
PWA installed from Safari	All of it	Lowest	Background audio works once playback is started from the page; Web Push since iOS 16.4; installed web apps are exempt from Safari's 7-day storage eviction *(unverified)*. No background downloads, no CloudKit, no App Store listing, storage caps in the low GB. Good enough to validate the product.

Option	Shared code with the web app	Effort	Trade-offs
Expo/React Native with shadow-cljs (`:react-native` target)	The `.cljc` core and most view logic if you use a React wrapper on both targets	Medium	Mature path; Podverse ships this way (in TypeScript). `react-native-track-player` for background playback and downloads; Expo Modules API to write a small Swift module for CloudKit. Krell is the alternative CLJS/RN tooling but shadow-cljs is the safer bet in 2026.
ClojureDart + Flutter	The `.cljc` core only (ClojureDart reads `.cljc` with the `:cljd` feature); UI is rewritten in Flutter	Medium to high	Excellent native audio stack (`just_audio`, `audio_service`), `icloud_storage` plugin. Still young tooling (limited REPL, fewer libraries; Malli not supported *(unverified)*). Pick this only if you want Flutter for its own sake.
Native Swift client, Clojure backend	None on the device	High	Best iOS citizen (CloudKit, background downloads, Live Activities). The Clojure library then lives entirely server-side. Reasonable if the app is thin and the backend owns feeds, search and sync.

Recommendation: PWA first, then the Expo/shadow-cljs shell when you need background downloads, CloudKit sync or App Store distribution. It keeps one language and one core and is the path with existing precedent.

9. Cloud syncing of OPML and playback state

9.1 What iCloud does and does not offer

- **Apple Podcasts' own library** is stored in a private CloudKit container. Third parties cannot read it; the only interchange is manual OPML export/import on macOS.
- **CloudKit private database** is the one iCloud facility usable by third-party apps on iOS and macOS. Records are limited to about 1 MB with larger blobs as `CKAsset`; storage counts against the user's iCloud quota, not yours; it needs a paid Apple Developer Program membership and a container.
- **CloudKit JS and CloudKit Web Services** let a web app use the same container. The user signs in with their Apple ID in an Apple-hosted popup and the page then reads and writes the private database. It needs an API token from the CloudKit console. The library has changed little since about 2018 but remains documented and supported *(unverified as of 2026)*. This is the fact that makes "iCloud sync between my iPhone app and my web app" feasible.
- **iCloud Drive documents** (the ubiquity container) and the **key-value store** (1 MB, 1024 keys) work for a native app but are not reachable from the web. An OPML file in iCloud Drive is a fine export target and a poor sync primitive.
- App Store guideline 4.8: if the app offers any third-party login, it must also offer Sign in with Apple or an equally private option.

9.2 Recommended design

1. Treat the subscription list and the playback state as two versioned documents (section 7.6), independent of any backend.
2. Implement CloudKit as one backend: one record per document for the simple version, or one record per subscription and per episode state for finer merges. On iOS this is a small Expo module in Swift; on the web it is CloudKit JS from ClojureScript.
3. Implement WebDAV as the second backend, which covers Nextcloud and most self-hosters, and gives Android users a route without an Apple ID.
4. Expose a gpodder-compatible API from the Clojure backend so existing apps (AntennaPod, Podcast Addict) can sync against your server, and adopt the Open Podcast API once it has a stable release.
5. Always offer plain OPML import and export. It is the lingua franca and what users will bring from Apple Podcasts, Overcast or Pocket Casts.

10. Facts to verify before committing

- Which namespace tags Apple Podcasts reads today beyond `transcript` (JSON `chapters` in particular).
- The current `podcast:value` attribute vocabulary (`lnaddress` recipients) and whether keysend is still the recommended method.
- CloudKit JS: that it still authenticates web users and grants private-database access in 2026, and the current API token setup.
- ClojureDart: current `.cljc` support and which cljc libraries load, if that route is considered.
- Safari's storage eviction rules for installed web apps and background audio behaviour on the current iOS.
- The exact PSP-1 tag list and the current Podcast Index per-tag adoption counts.

11. Suggested sequence

1. `core.feed` + `core.formats` with the fixture corpus from the Podcast Index dump; publish to Clojars.
2. `podcast.index` client and a JVM crawler with conditional GET; add the Podping consumer.
3. The Clojure backend and the ClojureScript PWA: subscribe, play, chapters, transcripts, OPML import/export.
4. `podcast.sync` with WebDAV first (easy to test), then CloudKit via CloudKit JS on the web.
5. Expo/shadow-cljs iPhone shell with the CloudKit Swift module and background downloads.
6. Optional `podcast.v4v` with NWC, if you want to participate in value-for-value.

Sources

- Podcast namespace repository: https://github.com/Podcastindex-org/podcast-namespace
- Namespace docs: https://podcasting2.org/docs/podcast-namespace (value tag page confirmed live on 2026-09-20; the tag index URL returned 404 at that time)
- Podcast Index API docs: https://podcastindex-org.github.io/docs-api/
- Podcast Index apps directory: https://podcastindex.org/apps
- Open Podcast API: https://openpodcastapi.org/ and https://github.com/OpenPodcastAPI/api-specs
- podcast-partytime: https://www.npmjs.com/package/@podverse/podcast-partytime
- feedsmith: https://github.com/macieklamberski/feedsmith and https://feedsmith.dev/reference/namespaces/podcast
- podcastdx-client: https://github.com/RyanHirsch/podcastdx-client
- Alby SDK / NWC: https://github.com/getAlby/js-sdk and https://guides.getalby.com/developer-guide/nostr-wallet-connect-api/building-lightning-apps/nwc-js-sdk
- podcast_search (Dart): https://pub.dev/packages/podcast_search
- Rome modules list: https://rometools.github.io/rome/Modules/index.html
- Apple Podcasts RSS requirements: https://podcasters.apple.com/support/823-podcast-requirements

- CloudKit JS: https://developer.apple.com/documentation/cloudkitjs